Sortieren

Aufgabe 1

Betrachten Sie den folgenden Algorithmus 1 HOARE-PARTITION[1]:

Algorithm 1 HOARE-PARTITION (A, p, r)

```
1:  x ← A[p]
2:  i ← p − 1
3:  j ← r + 1
4:  while true do
5:     repeat
6:        j ← j − 1
7:     until A[j] ≤ x
8:     repeat
9:        i ← i + 1
10:    until A[i] ≥ x
11:    if i < j then
12:       A[i], A[j] ← A[j], A[i]
13:    else
14:       return j
```

Bemerkung zu den endcheck-Schleifen: Bitte beachten Sie, dass die Bedingungen der „repeat - until“-Schleifen negiert werden müssten, falls statt der „repeat - until“-Schleifen entsprechende „do - while“-Schleifen verwendet werden würden. (Beispiel: $!(A[j] \leq x) = A[j] > x$)

Machen Sie sich anhand des Beispiels Aufgabe 3 aus Übungsblatt 6 klar, dass dieser Algorithmus die folgenden Eigenschaften hat:

1. HOARE-PARTITION funktioniert, wenn das Feld $A[p..r]$ mindestens zwei Elemente enthält.
2. HOARE-PARTITION teilt das Feld $A[p..r]$ in zwei Felder $A[p..j]$ und $A[j+1..r]$. Im ersten Feld sind alle Elemente kleiner oder gleich denen des zweiten Feldes.
3. Das Pivot Element ist in einem der beiden Teilfelder enthalten.

Überlegen Sie sich, wie man QUICKSORT umschreiben muss, um HOARE-PARTITION zu benutzen. (Tipp: Sie müssen die Indizes in Zeilen 3 und 4 von QUICKSORT (Alg. 5.7) adjustieren – es ist für das Verständnis auch hilfreich, den C-Code `quicksort.c` zu modifizieren.) Überlegen Sie sich, welcher der beiden Algorithmen „besser“ ist.

Aufgabe 2

Wie oft wird die Funktion QUICKSORT aus dem Skript im Worst- und Best-Case insgesamt aufgerufen, wenn das Array 3, 4, 5 oder 9 Elemente beinhaltet?

Aufgabe 3

Welchen Wert gibt die Funktion PARTITION[2] bei folgendem Array [0, 19, 10, 16, 21, 7, 11, 13] nach dem ersten Durchlauf zurück?

[1] Aus Cormen, 3rd Edition, Seite 185 – hier allerdings so umformuliert, dass unsere Konvention befolgt wird, die Laufindizes der Felder von 0 bis $n-1$ (an Stelle von 1 bis n) anzugeben.

[2] Mit PARTITION ist hier und im folgenden die Version aus der Vorlesung gemeint, nicht HOARE-PARTITION.

Aufgabe 4

Was passiert, wenn dem QUICKSORT Algorithmus ein aufsteigend sortiertes Array übergeben wird? Wird der Dreieckstausch (Zeilen 6 und 7) in der PARTITION Funktion trotzdem ausgeführt? Wenn ja, welche Werte werden getauscht?

Aufgabe 5

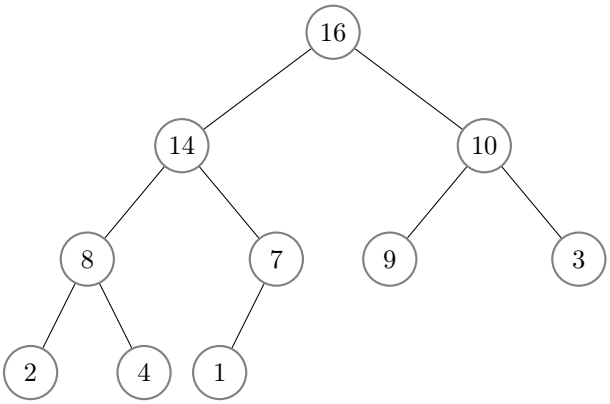
Dem QUICKSORT Algorithmus wird ein absteigend sortiertes Array übergeben. Was passiert? (Machen Sie sich das an einem Beispiel klar.)

Aufgabe 6

Folgendes Array ist gegeben A = [8, 2, 3, 4, 5, 9, 7]. Wie sehen die sortierten Arrays nach den folgenden Aufrufen aus: QUICKSORT(A, 0, 4) bzw. QUICKSORT(A, 2, 6)?

Aufgabe 7

Illustrieren Sie HEAPSORT an folgendem Beispiel (Schritt für Schritt):



Mit anderen Worten: Zeichnen Sie den Baum, wie er sich bei jedem Schritt verändert. Beachten Sie, wie die MAX-HEAP Eigenschaft verletzt und durch MAX-HEAPIFY wieder hergestellt wird.

Aufgabe 8

Zeichnen Sie den folgenden Heap $(27, 13, 11, 9, 8, 10, 4, 3, 5, 2)$ in Baum- und Feldnotation (analog zu Abbildungen 5.7 bis 5.9 im Skript).

Aufgabe 9

Sortieren Sie per Hand A = [2, 1, 8, 5, 4, 7, 0, 3, 9] nach dem HEAPSORT Algorithmus und veranschaulichen Sie jeden einzelnen Schritt Ihrer Lösung mittels der Baummethode wie oben.

Aufgabe 10

Testen Sie `heapsort.py` und ändern Sie die Datei so ab, dass

1. absteigend sortiert wird.
2. bei jedem rekursiven Aufruf Zwischenschritte ausgegeben werden.

Aufgabe 11

Implementieren Sie den Sortieralgorithmus HEAPSORT aus der Vorlesung in C. Dabei sollen Sie

mittels Einrücken eine Ausgabe in der Form der Baummethode erzeugen (Siehe Skript, Kapitel 5.3: "HeapSort").

Aufgabe 12

Zeigen Sie, dass ein Heap mit n Elementen die Höhe $\lfloor \lg n \rfloor$ hat. Dabei sei die *Höhe* definiert als die Anzahl der Kanten auf dem Weg von der Wurzel zu einem der "tiefsten" Blätter im Heap.

Aufgabe 13

Wo sitzt in einem MIN-HEAP das kleinste Element (Annahme: alle Elemente sind verschieden)?

Tipps

- Regelmäßige Teilnahme an den Übungen ist Voraussetzung für eine erfolgreiche Klausurteilnahme!
- Bilden Sie kleine Lerngruppen!
- Lesen Sie Literatur!
- Versuchen Sie auch die Programmieraufgaben!
- Stellen Sie Fragen!